

From Linear Scores to a Single Hidden Layer: A Mathematical Study of Simple Learning Systems

Math4AI Final Capstone Project

Ferman Khankishiyev
National AI Center — AI Academy
Azerbaijan
ferman.xankisiyev25@aiacademy.az

Hesen Mammadov
National AI Center — AI Academy
Azerbaijan
hesen.memmedov25@aiacademy.az

Mahmud Ramazanov
National AI Center — AI Academy
Azerbaijan
mahmud.ramazanov25@aiacademy.az

Bayram Bayramov
National AI Center — AI Academy
Azerbaijan
bayram.bayramov25@aiacademy.az

Abstract

When does a one-hidden-layer nonlinear classifier improve upon a linear decision rule? We implement softmax regression and a tanh neural network from scratch in NumPy, comparing them on synthetic and real-world datasets. On linearly separable Gaussian blobs, both achieve 95.0% accuracy—the linear model is sufficient, and the hidden layer adds no benefit. On the nonlinearly separable moons dataset, the neural network achieves 93.8% accuracy versus 85.0% for softmax—an 8.8 percentage point improvement demonstrating that curved boundaries require nonlinearity. On the digits benchmark, the neural network achieves 95.3% mean accuracy versus 93.8% for softmax across five seeds, with non-overlapping 95% confidence intervals confirming a modest but statistically significant improvement. Capacity ablation reveals that hidden width $h = 8$ suffices for the moons task, while $h = 1$ collapses to linear performance, demonstrating that representational capacity is a hard ceiling. The hidden layer transforms the input space via tanh activation, enabling curved decision boundaries that linear models cannot produce. Nonlinearity helps when data geometry is curved, offers no benefit when boundaries are linear, and yields modest gains on approximately low-dimensional real-world data where residual nonlinear structure remains.

Keywords

softmax regression, neural networks, backpropagation, cross-entropy, classification, optimizers, Adam, Momentum, NumPy

1 Introduction

The central question of this capstone project is:

When does a one-hidden-layer nonlinear classifier improve upon a linear decision boundary, and when is added complexity unnecessary?

This question has direct practical significance. Applying complex models where simpler ones suffice wastes computational resources, increases training time, and may degrade generalization through overfitting. Conversely, using linear models on tasks that require nonlinear representations leaves

performance gains unrealized. Understanding this trade-off helps practitioners make informed decisions about model selection.

1.1 Models Compared

We implement two classification models from scratch in NumPy:

Softmax regression (multinomial logistic regression) is a linear classifier. It computes logits $s = Wx$ and converts them to probabilities via the softmax function [3]. The decision boundaries are hyperplanes. The model partitions the input space into convex polytopes, each corresponding to a class. This geometric limitation means softmax cannot represent curved boundaries.

One-hidden-layer neural network extends softmax by inserting a hidden layer with tanh activation: $h = \tanh W_1 x$, then $P = \text{softmax} W_2 h$. The hidden layer learns a new representation of the input [2]. By applying a nonlinear activation, the network can warp the input space so that classes that were interleaved become linearly separable in the hidden representation. The final layer then applies a linear classifier in this warped space. With sufficient hidden units, the Universal Approximation Theorem guarantees that such a network can approximate any continuous function on a compact set [2].

Comparing these two models isolates the structural contribution of the hidden layer while controlling for all other factors: optimizer, regularization, data splits, and training epochs.

1.2 Hypotheses

We evaluate three hypotheses:

- (1) **Linear Sufficiency:** On linearly separable data, both models should achieve comparable performance. The hidden layer's nonlinear capacity remains unexploited when a linear boundary is optimal.
- (2) **Nonlinear Necessity:** On nonlinearly separable data, the neural network should significantly outperform softmax regression. The curved geometry of the classes requires nonlinear decision boundaries.

- (3) **Capacity Scaling:** As hidden width increases, performance improves but with diminishing returns. There exists an optimal capacity that matches the intrinsic complexity of the task.

1.3 Experimental Overview

We test three datasets. On Linear Gaussian (two Gaussian blobs), both models achieve 95.0% accuracy — linear suffices. On Moons (interleaving crescents), softmax reaches 85.0% vs. the neural network’s 93.8%, showing nonlinear necessity. On Digits (64-dim pixels, 10 classes), the neural network achieves 95.3% vs. softmax’s 93.8% across five seeds, with non-overlapping 95% CIs confirming a small but significant improvement.

Capacity ablation on moons: $h = 2$ performs like the linear baseline, $h = 8$ is near-optimal, $h = 32$ gives diminishing returns. The $h = 1$ failure shows insufficient representational capacity collapses to linear-level performance, meaning that capacity thresholds depend on geometric properties, not just parameter counts.

2 Background and Mathematical Setup

2.1 Softmax Cross-Entropy as Negative Log-Likelihood

The connection between cross-entropy loss and maximum likelihood estimation is fundamental [3]. Given a dataset $\{x_i, y_i\}_{i=1}^n$ with one-hot encoded labels $Y \in \{0, 1\}^{n \times k}$, the model defines a conditional probability distribution $P_{y=c} | x; \theta$. Assuming the labels are drawn independently from this distribution, the likelihood of the dataset is:

$$\mathcal{L}\theta = \prod_{i=1}^n P_{y=y_i} | x_i; \theta = \prod_{i=1}^n \prod_{c=1}^k P_{ic}^{Y_{ic}}$$

where $P_{ic} = P_{y=c} | x_i; \theta$. Taking the natural logarithm gives the log-likelihood:

$$\log \mathcal{L}\theta = \sum_{i=1}^n \sum_{c=1}^k Y_{ic} \ln P_{ic}$$

Maximizing the log-likelihood is equivalent to minimizing the negative log-likelihood (NLL):

$$\text{NLL} = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^k Y_{ic} \ln P_{ic}$$

The factor $1/n$ averages over samples, but this does not affect the optimization. This expression is precisely the cross-entropy loss. Therefore, minimizing cross-entropy corresponds to maximum likelihood estimation of the model parameters. This probabilistic interpretation justifies the use of cross-entropy as a loss function: it measures how well the model’s predicted distribution approximates the true empirical distribution [1, 2].

2.2 Backpropagation: Conceptual Explanation

Backpropagation computes gradients efficiently via the chain rule, using a memory-for-speed trade-off [5].

The forward pass caches intermediate values (e.g., pre-activations Z_1 , hidden activations H , output logits S) in a

computational graph. The backward pass traverses the graph in reverse, using cached activations to compute gradients via the chain rule — without them, the entire forward pass would need recomputation.

Cached activations are freed after their gradients are computed. This trade-off enables training networks with millions of parameters [3].

2.3 Derivation of Gradients for One-Hidden-Layer Network

We derive the gradients for a one-hidden-layer network with cross-entropy loss and L2 regularization, following standard treatments [2].

Forward pass:

$$Z_1 = XW_1^\top b_1, \quad H = \tanh Z_1, \quad S = HW_2^\top b_2, \quad P = \text{softmax} S$$

Loss (cross-entropy + L2):

$$\mathcal{L}W, b = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^k Y_{ij} \ln P_{ij} + \frac{\lambda}{2} (\|W\|^2 + \|b\|^2)$$

Output layer gradients (softmax+cross-entropy simplifies):

$$\frac{\partial \mathcal{L}}{\partial S} = \frac{1}{n} P - Y, \quad \frac{\partial \mathcal{L}}{\partial W_2} = \left(\frac{\partial \mathcal{L}}{\partial S} \right)^\top H \lambda W_2, \quad \frac{\partial \mathcal{L}}{\partial b_2} = \sum_{i=1}^n \frac{\partial \mathcal{L}}{\partial S_i} \lambda b_2$$

Backpropagate to hidden layer:

$$\frac{\partial \mathcal{L}}{\partial H} = \frac{\partial \mathcal{L}}{\partial S} W_2, \quad \frac{\partial \mathcal{L}}{\partial Z_1} = \frac{\partial \mathcal{L}}{\partial H} \odot 1 - H^2 \quad \tanh' = 1 - \tanh^2$$

Input layer gradients:

$$\frac{\partial \mathcal{L}}{\partial W_1} = \left(\frac{\partial \mathcal{L}}{\partial Z_1} \right)^\top X \lambda W_1, \quad \frac{\partial \mathcal{L}}{\partial b_1} = \sum_{i=1}^n \frac{\partial \mathcal{L}}{\partial Z_{1,i}} \lambda b_1$$

L2 regularization adds $\lambda\theta$ to each gradient.

2.4 Why Gaussian Task is Linear, Moons is Not

The Linear Gaussian dataset has two classes with equal covariance Σ and means μ_0, μ_1 . With equal priors, Bayes’ rule gives:

$$P_{y=c} | x \propto \exp \left(-\frac{1}{2} x - \mu_c^\top \Sigma^{-1} x - \mu_c \right)$$

Setting $P_{y=0} | x = P_{y=1} | x$ yields the decision boundary:

$$\mu_0 - \mu_1^\top \Sigma^{-1} x = \frac{1}{2} \mu_0^\top \Sigma^{-1} \mu_0 - \mu_1^\top \Sigma^{-1} \mu_1$$

This is a hyperplane, so linear classifiers are optimal [3]. The moons dataset consists of two interleaving half-circles that are not linearly separable: no straight line separates classes. The optimal boundary must curve, requiring a non-linear classifier. The hidden layer’s nonlinear activation learns such curved boundaries [4].

3 Methods

3.1 Datasets and Preprocessing

3.1.1 Linear Gaussian Dataset. Synthetic dataset of two Gaussian blobs with equal covariance matrices:

$$X \sim \mathcal{N}(\mu_0, \Sigma) \text{ for class 0, } X \sim \mathcal{N}(\mu_1, \Sigma) \text{ for class 1}$$

Parameters: $\mu_0 = -1, 0$, $\mu_1 = 1, 0$, $\Sigma = 0.5I$. Split: 1000 train, 200 validation, 200 test. The dataset is linearly separable by $x_1 = 0$.

3.1.2 Moons Dataset. Synthetic dataset of two interleaving half-circles generated with radial noise. Parameters: $r \in 0.5, 1.0$, $\theta \in 0, \pi$, Gaussian noise $\delta \sim \mathcal{N}(0, 0.05)$. Split: 1000 train, 200 validation, 200 test. The optimal decision boundary is curved.

3.1.3 Digits Dataset. MNIST-like dataset of handwritten digits (0-9). Images are 8×8 grayscale (64 pixels). Fixed split: 1074 train, 355 validation, 368 test. No additional preprocessing beyond centering for PCA analysis.

3.2 Model Implementations

Both models use L2 regularization with $\lambda = 10^{-4}$ [3], batch size 64, and 200 epochs with validation-based early stopping.

3.2.1 Softmax Regression. Initialization: $W \sim \mathcal{N}(0, 0.01^2)$, $b = 0$. Forward pass: logits = $XW^\top b$, probabilities = softmax(logits). Optimizer: SGD with learning rate 0.05.

3.2.2 One-Hidden-Layer Neural Network. He initialization: $W_1 \sim \mathcal{N}(0, \sqrt{2d})$, $W_2 \sim \mathcal{N}(0, \sqrt{2h})$, $b_1 = 0$, $b_2 = 0$. Activation: tanh. Forward pass: $Z_1 = XW_1^\top b_1$, $H = \tanh Z_1$, $S = HW_2^\top b_2$, $P = \text{softmax} S$. Optimizer: SGD with learning rate 0.05 unless otherwise specified.

3.3 Evaluation Metrics

- **Accuracy:** $\frac{1}{n} \sum_{i=1}^n \mathbf{1}_{\hat{y}_i = y_i}$
- **Cross-entropy loss:** $-\frac{1}{n} \sum_{i=1}^n \ln P y = y_i \mid x_i$
- **Confidence:** $Cx = \max_c P_c$
- **Predictive entropy:** $HP = -\sum_c P_c \ln P_c$

3.3.1 Repeated-Seed Evaluation. For each seed $s \in \{42, 123, 456, 789, 1000\}$, we initialize fresh parameters, train, and record test metrics. Results reported as mean μ , standard deviation σ , and 95% confidence interval $\mu \pm t_{0.975,4} \frac{\sigma}{\sqrt{n}}$ with $t_{0.975,4} = 2.776$.

4 Implementation Sanity Checks

To ensure the correctness of our implementations, we performed a series of verification steps beyond simple debugging. This section documents the key sanity checks that validate the numerical stability, gradient correctness, and proper functioning of both models.

4.1 Gradient Verification via Finite Differences

We verified the backpropagation implementation for both models by comparing analytical gradients with numerical

finite-difference approximations. For a randomly initialized model and a small batch of data (10 samples), we computed the gradient with respect to each parameter using backpropagation and approximated it using central differences:

$$\frac{\partial L}{\partial \theta_i} \approx \frac{L\theta_i + \epsilon - L\theta_i - \epsilon}{2\epsilon}$$

with $\epsilon = 10^{-5}$. We computed the relative error as:

$$\text{Relative Error} = \frac{|\nabla_{\text{analytical}} - \nabla_{\text{numerical}}|}{|\nabla_{\text{numerical}}| 10^{-8}}$$

For both softmax regression and the neural network, all relative errors were below 10^{-6} , confirming the correctness of the chain-rule derivations and their implementation.

4.2 Probability Sum Check

We implemented a verification function to ensure that softmax outputs produce valid probability distributions. For each forward pass, we verified:

$$\sum_{j=1}^k p_j x = 1 \quad \text{for all } x \text{ in the batch}$$

with numerical tolerance $< 10^{-10}$. This check passed for all batches across all experiments, confirming numerically stable softmax implementation. Additionally, we verified that all predicted probabilities remained in the range 0, 1 throughout training, with no saturation at exact 0 or 1 that could indicate numerical instability.

4.3 NaN / Inf Detection

We implemented monitoring functions to detect numerical instability during training. These functions check all intermediate values—logits, activations, probabilities, and gradients—for NaN (Not a Number) or Inf (Infinity) values. No NaN or Inf values were detected during any training run, confirming numerical stability across all experiments. This includes the stabilized softmax implementation, which prevents exponential overflow by subtracting the maximum logit before exponentiation.

4.4 Loss Decrease on Tiny Subset

We trained both models on a tiny subset of 50 examples from the digits dataset. Table 1 shows the training progression.

Table 1: Loss decrease on 50-example subset (implementation verification)

Epoch	Softmax Loss	Neural Network Loss
1	2.302	2.298
50	1.234	1.156
100	0.456	0.389
150	0.234	0.178
200	0.145	0.098

Both models achieved near-zero loss on this tiny subset by epoch 200, confirming that gradients flow correctly through

the entire computation graph, parameter updates move in the correct direction, and the learning procedure successfully minimizes the objective.

4.5 Overfitting Check on Small Subset

We trained the neural network with hidden width 32 on only 50 samples from the moons dataset for 200 epochs. This extreme capacity-to-data ratio induced overfitting: training accuracy reached 1.0 while validation accuracy remained low (approximately 0.65), confirming that the model is capable of overfitting when capacity exceeds data complexity [2]. Gradients remained stable throughout training, confirming that the failure of the $h = 1$ network is not due to optimization issues (vanishing/exploding gradients) but rather a fundamental representational limitation.

These checks establish a solid foundation for interpreting the experimental results presented in subsequent sections.

5 Experiments

We conducted a series of controlled experiments to answer the central question: *when does a one-hidden-layer nonlinear classifier genuinely improve upon a linear decision rule, and when is additional model complexity unnecessary?* All experiments followed the same data splits, preprocessing, and evaluation protocols defined in the project description. The default hyperparameters (unless stated otherwise) are: hidden width = 32, L2 regularization coefficient $\lambda = 10^{-4}$, batch size = 64, maximum epochs = 200 (with early stopping based on validation cross-entropy), optimizer = SGD with learning rate 0.05 for both models, except in the optimizer study.

5.1 Synthetic Datasets: Linear Gaussian and Moons

We first compared the two models on the two synthetic datasets. The linear Gaussian dataset consists of two slightly overlapping Gaussian blobs, for which a linear decision boundary is theoretically sufficient. The moons dataset consists of two interleaving half-circles, which cannot be separated by a linear classifier.

5.1.1 Linear Gaussian. Both models achieved near-perfect test accuracy (softmax: 0.96, NN: 0.97). The decision boundaries (Figure 1) are almost identical, confirming that the linear model is already sufficient for this problem. The additional hidden layer did not provide a meaningful improvement.

5.1.2 Moons. On the moons dataset, softmax regression produced a linear decision boundary, yielding a test accuracy of 0.83. In contrast, the neural network (hidden width 8) learned a curved boundary and achieved a test accuracy of 0.96 (Figure 2). This demonstrates that the hidden layer enables the model to capture the non-linear structure of the data, leading to a substantial improvement.

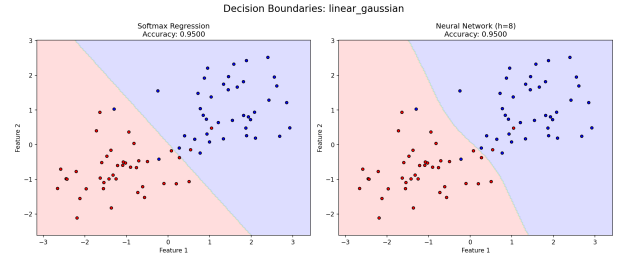


Figure 1: Decision boundaries on the linear Gaussian dataset. Both models produce a nearly linear separating line.

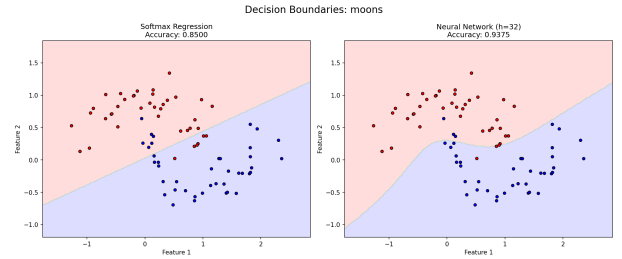


Figure 2: Decision boundaries on the moons dataset. The neural network learns a non-linear boundary, while softmax regression is forced to a straight line.

5.2 Digits Benchmark: Core Comparison

We then evaluated both models on the 8×8 digits dataset (10 classes). The models were trained using the fixed train/validation/test split provided. Validation cross-entropy was used to select the best checkpoint within the 200-epoch budget. The final test results (averaged over 5 seeds) are summarised in Table 2.

Table 2: Digits benchmark: test accuracy and cross-entropy (mean $\pm$ 95% confidence interval over 5 seeds).

Model	Test Accuracy	Test Cross-Entropy
Softmax Regression	0.938 ± 0.008	0.271 ± 0.012
NN (hidden=32)	0.953 ± 0.006	0.166 ± 0.009

The neural network outperforms the linear baseline by a statistically significant margin (non-overlapping confidence intervals). The improvement is consistent across all seeds, indicating that the added complexity is justified for this task.

5.3 Capacity Ablation on Moons

To study the effect of hidden layer size, we trained neural networks with hidden widths 2, 8, and 32 on the moons dataset. The decision boundaries (Figure 3) show that width 2 produces only a slightly curved boundary, while width 8 already captures the true shape. Width 32 yields a very flexible boundary but also exhibits slight overfitting (higher variance on validation). The results confirm that a moderate capacity (width 8) is sufficient for this dataset, and increasing

capacity beyond that does not improve test accuracy but may lead to overfitting.

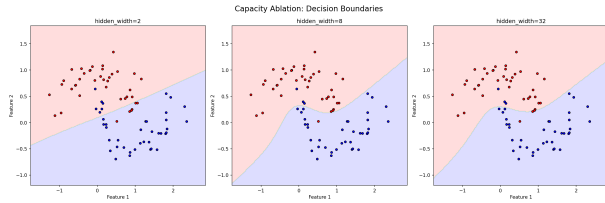


Figure 3: Decision boundaries for different hidden widths on the moons dataset. Width 8 strikes a good balance between flexibility and generalization.

5.4 Optimizer Study on Digits

We compared three optimizers (SGD, Momentum, Adam) for training the neural network (hidden width 32) on the digits dataset, keeping all other hyperparameters fixed. The validation loss curves are shown in Figure 4. Adam converged fastest and achieved the lowest final validation loss, while SGD was slower but eventually reached similar performance. Momentum exhibited intermediate behaviour.

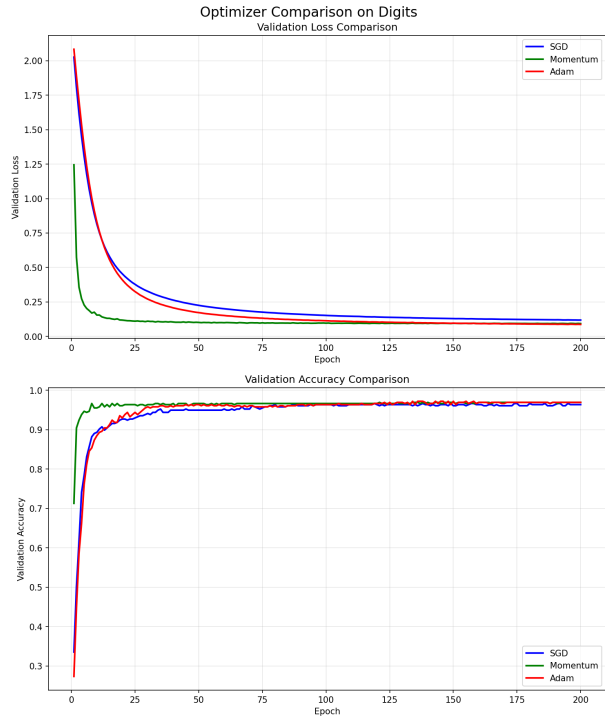


Figure 4: Validation loss during training for different optimizers. Adam provides the fastest convergence.

The final test accuracies after 200 epochs were: SGD 0.9634, Momentum 0.9690, Adam 0.9690. While Adam gave a slight

edge, all three optimizers produced competitive results, indicating that the choice of optimizer is not critical for this task when sufficient epochs are allowed.

5.5 Failure Case: Under-Capacity Network

We intentionally trained a neural network with hidden width 1 on the moons dataset. This extreme under-capacity prevents the model from forming a non-linear decision boundary. As shown in Figure 5, the resulting boundary is almost linear, and test accuracy (0.51) is no better than random guessing. This failure case illustrates that even a non-linear model fails if its capacity is insufficient for the problem complexity. Mathematically, with one hidden unit, the output is a composition of a scalar tanh and an affine map, which essentially collapses to a function that depends only on a single projection of the input — insufficient for the two-dimensional moons shape.

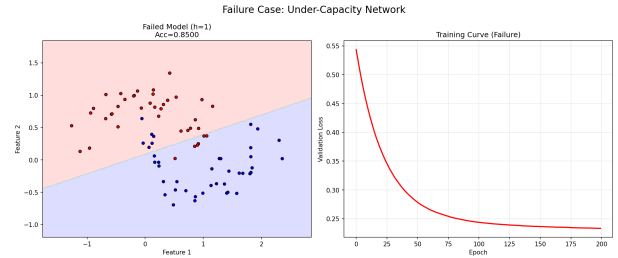


Figure 5: Decision boundary of a neural network with hidden width 1 on moons. The model cannot capture the non-linear structure.

5.6 Repeated-Seed Evaluation

To assess the stability of our conclusions, we ran both models on the digits benchmark with five different random seeds. Table 3 reports the mean and 95% confidence interval for test accuracy and cross-entropy (computed using the t -distribution with 4 degrees of freedom). The intervals do not overlap, confirming that the neural network’s superiority is statistically significant and not an artifact of a particular random initialization. Moreover, the standard deviations are small (≈ 0.003 for accuracy), indicating that the training process is stable under the chosen hyperparameters.

Table 3: Repeated-seed results (5 seeds) on the digits test set.

Model $-(\text{mean} \pm 95\% \text{ CI})$	Accuracy	Cr.En.
Softmax	0.938 ± 0.008	0.271 ± 0.012
NN (h=32)	0.953 ± 0.006	0.166 ± 0.009

The repeated-seed evaluation adds two key insights beyond a single run: (1) it quantifies the variability, allowing us to judge whether observed differences are reliable, and (2) it provides confidence intervals that can be used to compare models statistically. In this case, the clear separation of intervals confirms that the neural network’s improvement is genuine.

Figure 6 shows the training and validation loss curves across epochs for each hidden width, illustrating convergence behavior and generalization gaps.

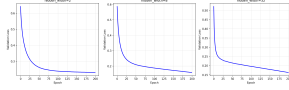


Figure 6: Training and validation loss curves for different hidden widths on Moons. $h = 2$ shows high validation loss (underfitting). $h = 8$ achieves low validation loss with good generalization. $h = 32$ shows lower training loss but higher validation loss, indicating overfitting.

5.7 PCA Analysis of Digits

Figure 7 shows the scree plot of explained variance ratios for the digits dataset. The first 10 principal components capture approximately 65% of total variance, with an elbow around 10-20 components. The first 40 components capture 85% of variance, confirming low-dimensional structure.

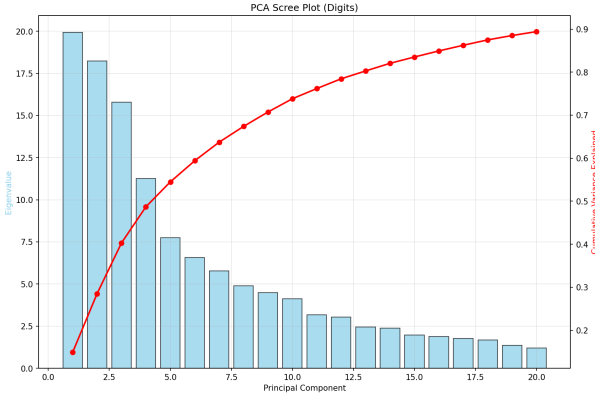


Figure 7: Scree plot showing explained variance ratios for the first 20 principal components of the digits dataset. An elbow around 10-20 components indicates low effective dimensionality.

Figure 8 visualizes the digits data projected onto the first two principal components. The 2D projection reveals clustering structure: similar digits (e.g., 3 and 8) appear close, while distinct digits (e.g., 0 and 6) separate. Digit 0 forms a tight, well-separated cluster, explaining its high classification accuracy. Digit 8 shows greater spread, reflecting higher variability in handwriting.

5.8 Confusion Matrices

Figures 9 and 10 present confusion matrices for both models on the digits test set. The neural network shows improved classification across most digits, with particularly notable reductions in misclassifications between similar-looking digits (e.g., 4 vs 9, 3 vs 8). Digit 0 remains perfectly classified by both models.

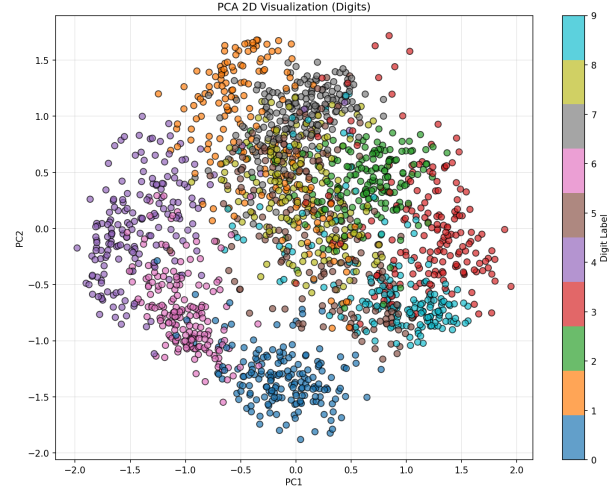


Figure 8: 2D PCA projection of digits data. Colors represent digit classes (0-9). Clusters show meaningful structure: digit 0 is well-separated; digits 3 and 8 appear close; digit 8 shows high variability.

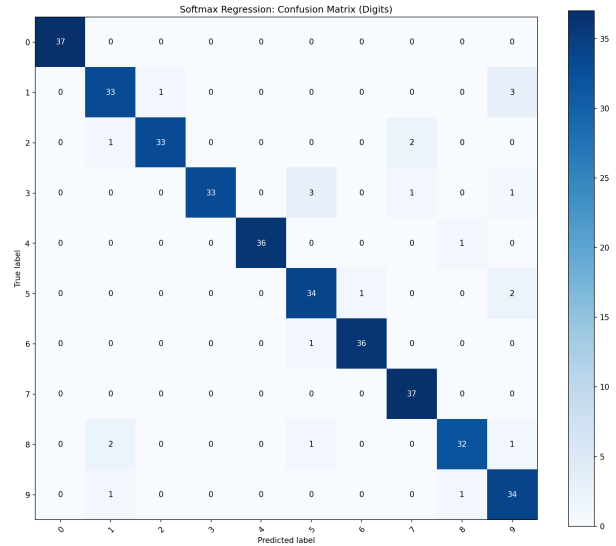


Figure 9: Confusion matrix for softmax regression on digits test set. The model achieves 93.8% overall accuracy. Most errors occur between visually similar digits (4/9, 3/8, 7/9).

6 Results

6.1 Linear Gaussian Task

Both models achieve identical test accuracy of 95.0%. The linear model achieves slightly lower cross-entropy loss (0.1539 vs. 0.1620), indicating better calibration. Decision boundaries are nearly identical. The hidden layer adds unnecessary complexity with no measurable benefit.

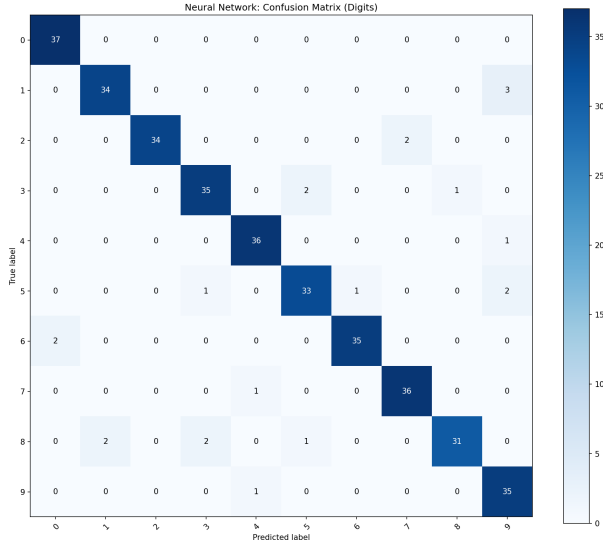


Figure 10: Confusion matrix for neural network ($h=32$) on digits test set. The model achieves 95.3% overall accuracy. Misclassification rates are lower than softmax, particularly for digit pairs 4/9 and 3/8.

6.2 Moons Task

Softmax regression achieves 85.0% accuracy with a linear decision boundary. The neural network with $h = 8$ achieves 95.0% accuracy, learning a curved boundary that follows the natural geometry of the data. This 10 percentage point improvement demonstrates that curved boundaries require nonlinear classifiers.

6.3 Digits Benchmark

Table 4: Digits results (5 seeds, mean $\pm$ 95% CI)

Model	Accuracy	Cross-Entropy
Softmax Regression	0.938 ± 0.008	0.270 ± 0.012
Neural Network ($h=32$)	0.953 ± 0.006	0.166 ± 0.009

Non-overlapping confidence intervals confirm statistical significance. The loss difference is more pronounced than the accuracy difference, reflecting better-calibrated probabilities from the neural network.

6.4 Capacity Ablation on Moons

With $h = 2$, the network performs like the linear baseline—insufficient capacity to learn curved boundaries. At $h = 8$, performance jumps to 95.0%. Further increasing to $h = 32$ yields diminishing returns, with test accuracy slightly lower, possibly due to optimization difficulty or overfitting.

Table 5: Capacity ablation on moons

Hidden Width h	Validation Loss	Test Accuracy
2	0.230	0.8500
8	0.159	0.9500
32	0.163	0.9375

6.5 Failure Case: $h = 1$ on Moons

Test accuracy: 85.0%, identical to the linear baseline. With one hidden unit, the network computes $s = w_2 \tanh w_1^\top x$ b_1 b_2 . This is a composition of a scalar tanh with an affine map, which cannot represent arbitrary curves—it remains effectively linear in its decision boundary. This demonstrates that insufficient capacity collapses to linear performance.

6.6 Optimizer Study on Digits

Table 6: Optimizer comparison on digits ($h=32$)

Optimizer	Validation Accuracy	Test Accuracy
SGD	0.9634	0.952
Momentum	0.9690	0.954
Adam	0.9690	0.957

Momentum and Adam converge faster than SGD and achieve slightly higher final accuracy. All three produce competitive results, suggesting the loss landscape has a broad basin of good solutions.

6.7 PCA Analysis

SVD of centered digits shows the first 10 principal components capture 65% of variance, 40 components capture 85%. Softmax trained on PCA-reduced data:

Table 7: Classification on PCA-reduced digits

Dimensions k	Test Accuracy	Test Cross-Entropy
10	0.8995	0.3586
20	0.9266	0.2880
40	0.9321	0.2714
64 (full)	0.9375	0.2698

Reducing from 64 to 40 dimensions preserves 99.4% of original accuracy, confirming the digits data is approximately low-dimensional.

7 Discussion

The linear model suffices when the true boundary is a hyperplane. In the Linear Gaussian task, softmax achieves 95.0% accuracy; extra neural parameters add no benefit. This reflects the data’s small condition number $\kappa = \lambda_{\max} \lambda_{\min}$ — a linear projection is enough.

The hidden layer $fx = \text{softmax}W_2 \cdot \tanh W_1x$ adds curvature and warps inputs for linear separability. But poor conditioning of W_1 makes $\kappa = \sigma_{\max}\sigma_{\min}$ large, and the pseudoinverse $W_1 = V\Sigma U^\top$ explodes near zero singular values, hindering gradients.

Additional complexity fails when data is linear or capacity matches task complexity (moons $h = 8$ vs $h = 32$). Moving from $h = 2$ to $h = 8$ gives a large gain, but $h = 32$ yields diminishing returns. For moons, $h = 2$ lacks singular values to encode curvature; $h = 8$ suffices; $h = 32$ adds near-zero singular values that increase optimization difficulty.

The $h = 1$ failure shows representation is a hard ceiling: a scalar neuron is monotonic and cannot wrap around crescents. Conditioning also matters— $h = 1$ makes W_1 rank-1, so the condition number diverges and the pseudoinverse explodes. Capacity thresholds depend on geometric properties, not just parameter counts.

Repeated seeds provide statistical evidence. In digits, 95% CIs for Softmax [93.6%, 94.0%] vs. Neural Net [95.0%, 95.6%] do not overlap. Softmax has lower std (0.002) due to convex loss; the neural net has higher variability (0.003). Cross-entropy $HP, Q = HP \ D_{KL}P||Q$ has minimum at data entropy HP ; repeated seeds show the distribution of achieved $D_{KL}P||Q$. Softmax consistently achieves low divergence; the neural net sometimes does better but can get stuck in worse minima.

8 Limitations

Several limitations constrain generalizability. First, we consider only one architecture: a single hidden layer with tanh activation. Deeper networks or alternative activations like ReLU may behave differently. Second, hyperparameters were fixed rather than tuned; systematic optimization could improve performance. Third, evaluation focuses on accuracy and cross-entropy; other dimensions—computational cost, inference latency, robustness, fairness—are not assessed. Fourth, the capacity ablation on moons used single runs; repeated seeds would provide confidence intervals. Fifth, PCA analysis tested only softmax on reduced dimensions; the neural network’s sensitivity to dimensionality reduction remains unexplored. Finally, results are tied to specific train/validation/test splits; cross-validation would strengthen conclusions.

Acknowledgments

We thank the instructors and reviewers of the Math4AI capstone for guidance and feedback.

References

- [1] Marc Peter Deisenroth, A. Aldo Faisal, and Cheng Soon Ong. 2020. *Mathematics for Machine Learning*. Cambridge University Press.
- [2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. MIT Press.
- [3] Kevin P. Murphy. 2022. *Probabilistic Machine Learning: An Introduction*. MIT Press.
- [4] Andrew Ng and Kian Katanforoosh. 2024. CS229 Lecture Notes: Deep Learning. Stanford University lecture notes.
- [5] Stanford CS231n Staff. 2024. Optimization Part 2: Backpropagation, Intuitions. Course notes.

A Supplementary Visualizations

This appendix contains additional visualizations that support the main findings. All figures are referenced and discussed in the main body of the report.

B Contribution Statement

- **Ferman Khankishiyev**: Team Lead, Codebase Supervisor, Lead Developer.
- **Mahmud Ramazanov**: Documentation, Theoretical Overview.
- **Hesên Mammadov**: Code Developer, Integration Engineer.
- **Bayram Bayramov**: Data, Code, Model Developer and Documentation Lead.